Unified Document Viewer

About me: **Hien Giang**, Backend Developer

I choose: Scenario D - Backend

The problem

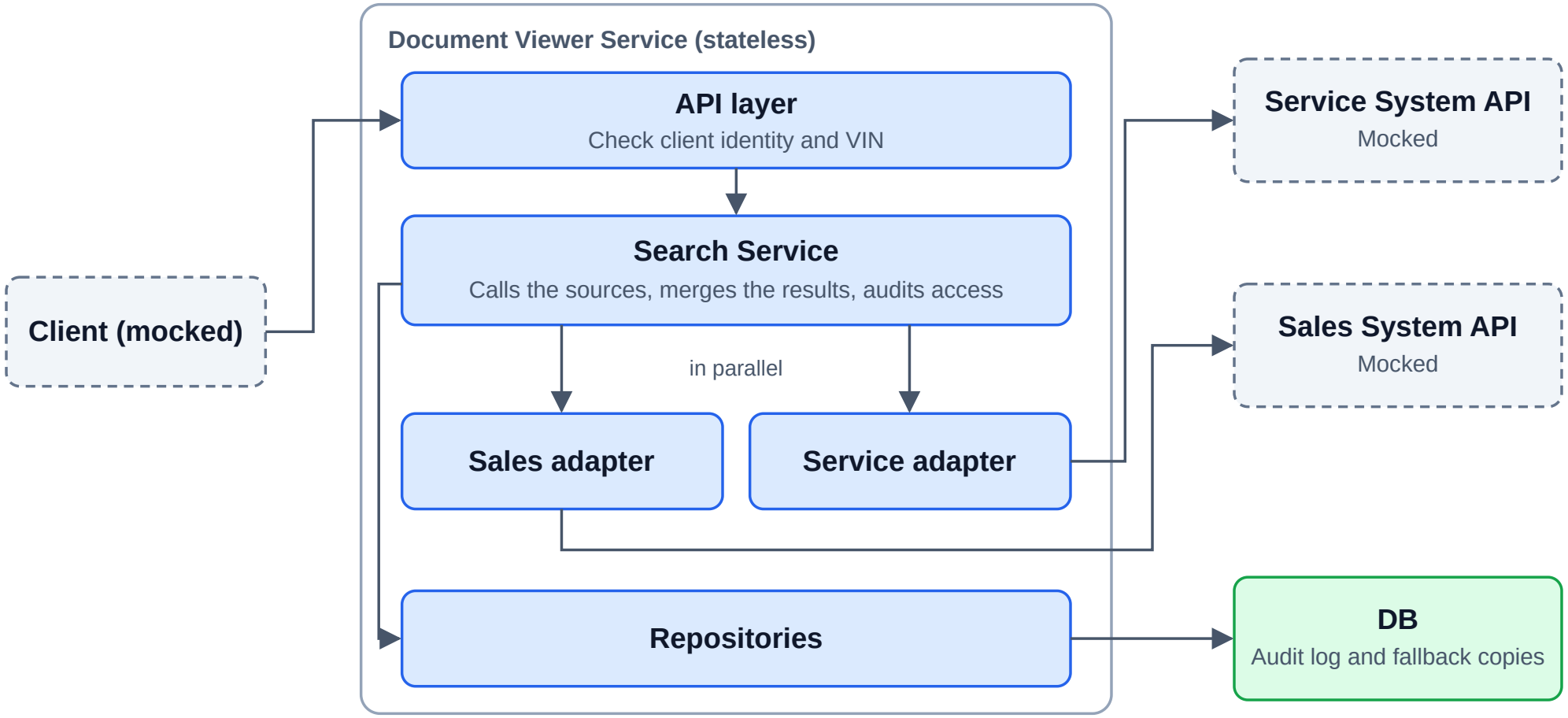
Users want to get a list of documents related to their VIN (Vehicle Identification Number)

- The backend must call external systems **in parallel**

Core assumptions

Assumption	Consequences
We can serve stale docs, but only up to 24 hours	Need a persistence engine to store fresh documents in successful calls A mechanism to clear stale docs Document must have state (NEW, STALE)
VINs can be personal data (linked to a person)	Require client identity and every access is audited. VINs are also masked everywhere
The external APIs return different shapes	Cannot express all schemas, only send document URLs back to client

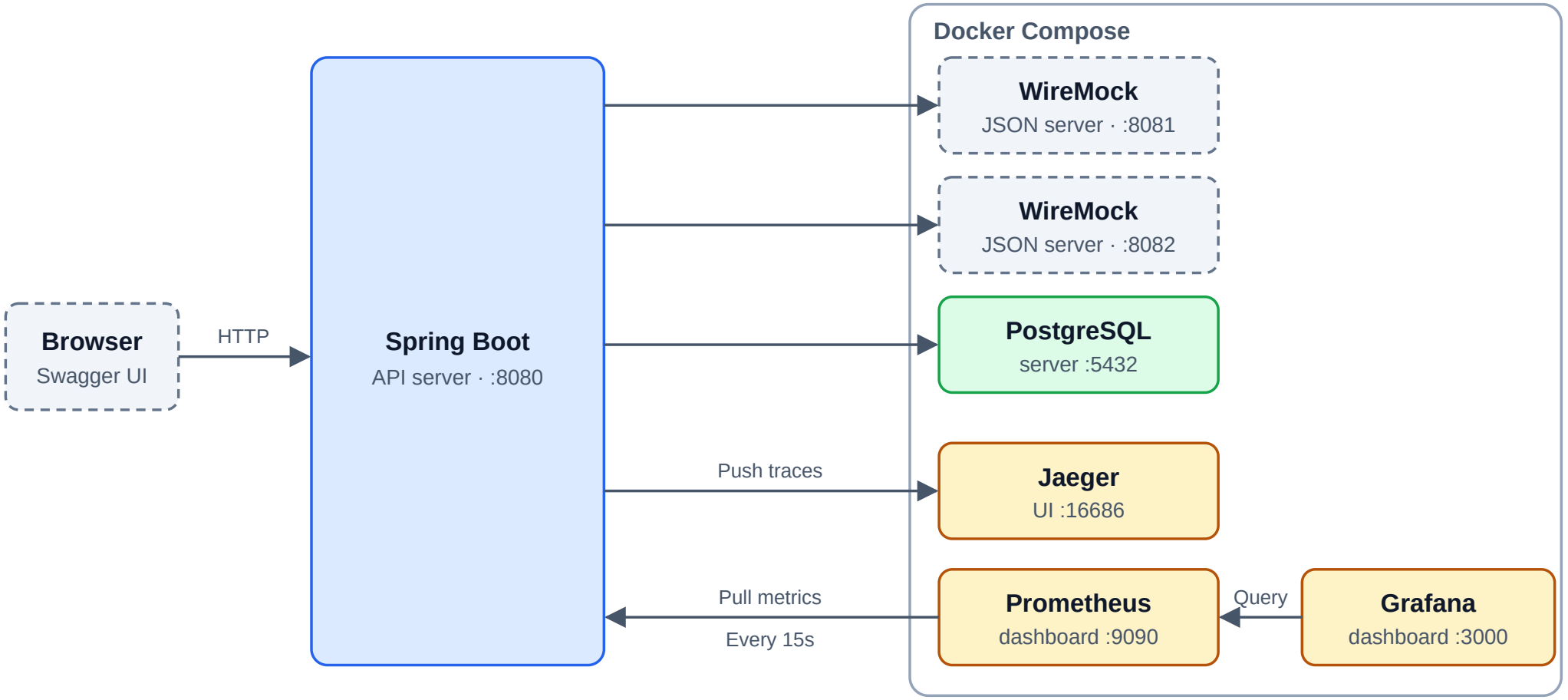
System Architecture



Why this Architecture?

Part	The problem it solves
An adapter per source	A new data source only needs a new adapter, without changing the architecture
Snapshot copies	Fairly new documents can still be served when the external services are down
Audited access	Suspicious access can be traced, because VIN-related documents are sensitive

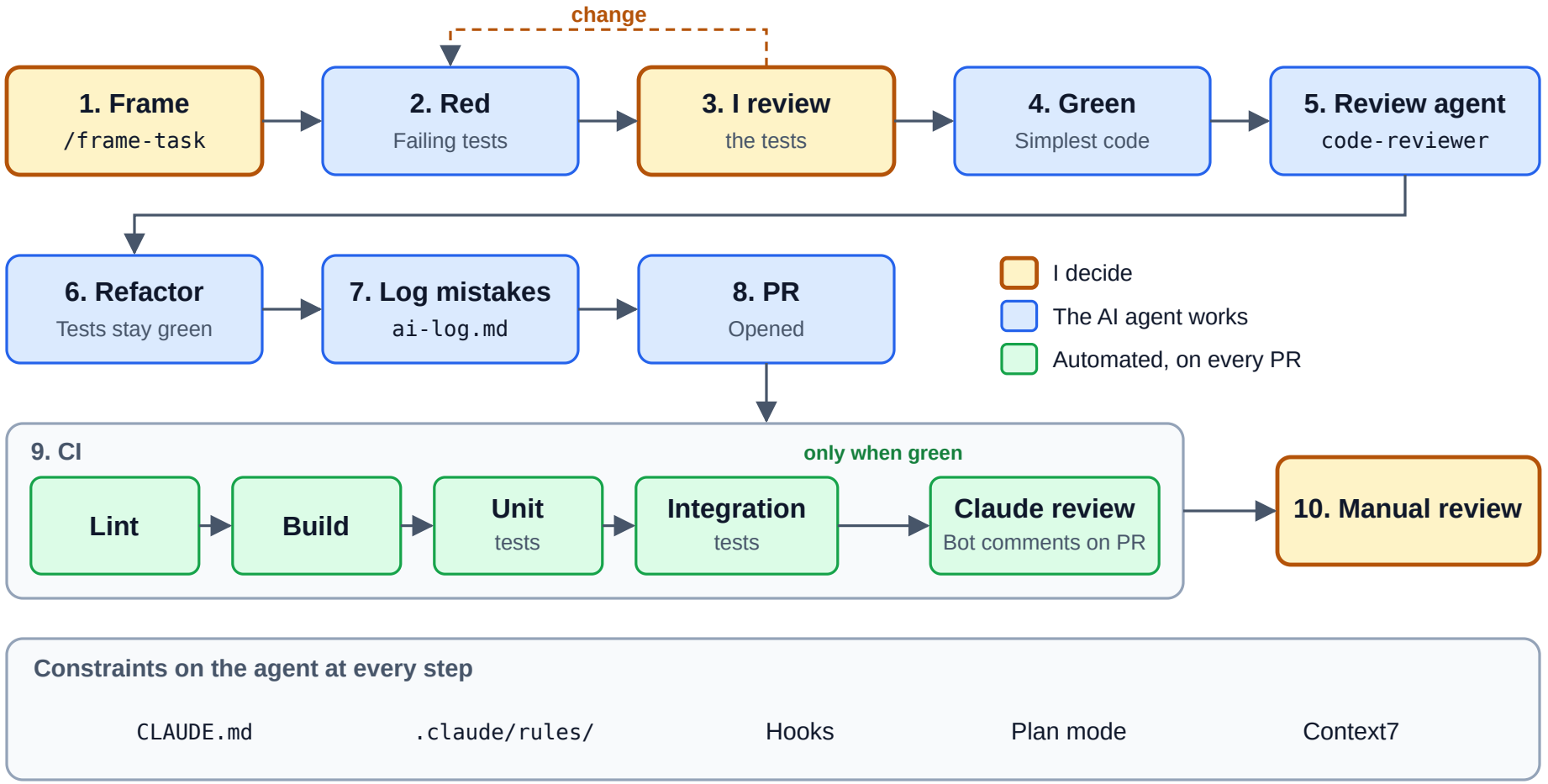
Application Architecture



Implementation highlights

Highlight	Why
HTTP timeout and service deadline configured	Service limits user wait HTTP timeout closes the connection between 2 hosts
Virtual threads enabled for API calls	Non-blocking I/O
A circuit breaker per source	Only failed sources are blocked, healthy sources keep serving
Retry	Can get a successful response after transient errors
Observability: metrics, logs and traces are enabled	Monitor the system

AI story: my harness



AI story: four layers of verification

Layer	What it caught
Automated tests	Agent generated buggy retry logic
`code-reviewer` agent	A slow search could overwrite data of a fast search, with all tests green
Claude in CI	Recommended DB configuration for connection timeout and query timeout
Manual review	Claude is over-engineered Claude closed a PR without asking

28 mistakes logged in `docs/ai-log.md`, each with how it was caught and fixed

Live demo 1: Happy path

Set up: None

Request

- VIN: `WVWZZZ1JZXW000001`
- Headers: `X-User-Id: user-1`, `X-Dealership-Id: dealer-1`

Expectation

- **API Response: 200** with 4 documents, each labelled with its source
- **Tracing:** External APIs are called at the same time, then the access is saved to the audit log

Live demo 2: Invalid caller identity

Set up: None

Request

- VIN: `WVWZZZ1JZXW000001`
- Headers: `X-User-Id` and `X-Dealership-Id` left blank

Expectation

- **API Response: 401**: the request is rejected
- **Tracing:** Only the incoming request. Sales and Service aren't called, and nothing is audited

Live demo 3: Retry

Set up: None. The Service mock always returns 503 for this VIN

Request

- VIN: `WVWZZZ1JZXW000003`
- Headers: `X-User-Id: user-1`, `X-Dealership-Id: dealer-1`

Expectation

- **API Response: 200** with the Sales documents only. The list is marked incomplete, and Service is shown as unavailable because it returned an error
- **Tracing:** Service is called twice: the first call fails, then one retry, which also fails

Live demo 4: Fallback copy

Set up: Stop the Service mock: `docker compose stop service-api`

Request

- VIN: `WVWZZZ1JZXW000001`
- Headers: `X-User-Id: user-1`, `X-Dealership-Id: dealer-1`

Expectation

- **API Response: 200**, with Service's documents taken from the copy saved in test 1. Service is marked out of date, with the time the copy was saved (`SERVICE`: `STALE`)
- **Tracing:** The call to Service fails, then the saved copy is read from the database

Live demo 5: Circuit breaker

Set up: Service is still stopped. On the Swagger tab, press `F12`, open Console and run this script. It sends 15 searches, which opens the breaker

```
for (let i = 0; i < 15; i++) {
  const r = await fetch('/api/v1/vehicles/WVWZZZ1JZXW000001/documents',
    { headers: { 'X-User-Id': 'user-1', 'X-Dealership-Id': 'dealer-1' } });
  console.log(i + 1, r.status);
}
```

Request (straight after the script)

- VIN: `WVWZZZ1JZXW000001`
- Headers: `X-User-Id: user-1`, `X-Dealership-Id: dealer-1`

Expectation

- **API Response: 200** in milliseconds, with Service's saved copy. The reason says the breaker is open, so Service wasn't called (`SERVICE`: `STALE`, `CIRCUIT_OPEN`)
- **Tracing:** Service isn't called at all, because the breaker is open. The saved copy is still read

What I learned, and the challenges

What I learned

- Green tests can still be false, there are blind spots we cannot see
- AI tends to over-engineer. Refactoring is a must

Challenges

- **The AI used advanced framework features that I did not know**. I used Context7 MCP server to minimize guessing
- **The AI created circular dependencies in the architecture.** Solved by a written one-way dependency rule

Thank you

Repository: github.com/hinn1411/document-viewer

- Design: `github.com/hinn1411/document-viewer/blob/main/docs/system-design-document.md`
- README: `github.com/hinn1411/document-viewer/blob/main/README.md`